SAMSUNG SW AI ACADEMY FOR YOUTH

함께가요 미래로!
Enabling People

SSAFY

# Django

## Authentication System 01

## Python

SAMSUNG SW AI ACADEMY FOR YOUTH

함께가요 미래로!
Enabling People

# 목 차


## Cookie & Session

- HTTP
- 쿠키
- 세션

## Django Authentication System

## Custom User model

- User model 대체하기

## Login

SAMSUNG SW AI ACADEMY FOR YOUTH

함께가요 미래로!
Enabling People

# 목 차

## Template with Authentication data

## 참고

- AbstractUser class
- 쿠키의 수명
- 쿠키와 보안
- Django에서의 세션 관리
- AuthenticationForm 내부 코드
- User 모델 대체하기 Tip

# 학습 시작



## 매번 방문하는 웹사이트, 어떻게 나를 기억하고 반겨줄까요?

1. 우리가 즐겨 찾는 쇼핑몰이나 SNS에 접속할 때, 아이디와 비밀번호를 한 번만 입력하면 다음 방문 시에는 자동으로 로그인 상태가 유지됩니다.
2. 하지만 웹은 [ ] 특징을 가지고 있습니다. 추가로 [ ] 특징도 가지고 있습니다.페이지를 이동할 때마다 우리는 사실 '처음 보는 사람' 취급을 받게 됩니다.
3. 그렇다면 웹사이트는 어떻게 이 문제를 해결하고 우리를 기억해 주는 걸까요?

- 이번 시간에는 웹사이트가 사용자를 기억하는 핵심 열쇠인 **쿠키(Cookie)**와 **세션(Session)**의 비밀을 파헤쳐 봅니다.
- Django가 제공하는 강력한 인증 시스템(Authentication System)을 활용하면, 이 복잡한 과정을 단 몇 줄의 코드로 안전하고 쉽게 구현할 수 있습니다.

# 학습 목표



- ✓ HTTP의 무상태 특징을 이해하고 인증의 필요성을 안다.
- ✓ 쿠키와 세션의 동작 원리 및 데이터 저장 방식 차이를 안다.
- ✓ Django 기본 User 모델의 한계를 알고 커스텀 모델을 만든다.
- ✓ AbstractUser를 상속받고 AUTH_USER_MODEL을 설정한다.
- ✓ AuthenticationForm과 login 함수로 로그인 기능을 구현한다.
- ✓ 로그인 시 세션이 생성되고 쿠키로 ID가 전달됨을 안다.
- ✓ request.user 객체로 로그인 상태에 따라 다른 화면을 표시한다.

SAMSUNG SW AI ACADEMY FOR YOUTH

함께가요 미래로!
Enabling People

# Cookie & Session

SAMSUNG SW AI ACADEMY FOR YOUTH

함께가요 미래로!
Enabling People

# HTTP

## HTTP

# HTTP

**HyperText Transfer Protocol**

HTML 문서와 같은 리소스들을
가져올 수 있도록 해주는 규약

(웹에서의 모든 데이터 교환의 기초)

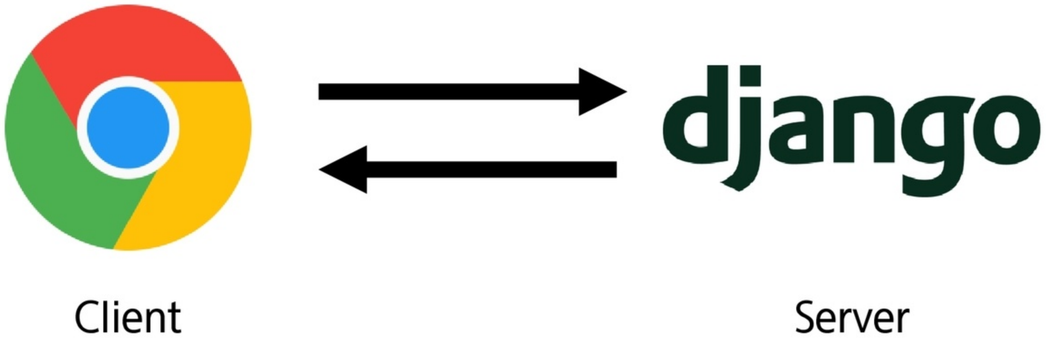
HTTP는 웹 브라우저와 서버가 서로 대화하기 위해 사용하는 공통 언어 또는 **약속**입니다.

**브라우저**가 '이 페이지 보여줘'라고 요청(Request)을 보내면,
**서버**는 그에 맞는 HTML, 이미지 등을 응답(Response)으로 보내주는 방식으로 동작합니다.

우리가 웹사이트를 보고, 이미지를 다운로드하고, 데이터를 주고받는 모든 과정이 바로 이 HTTP라는 규칙 위에서 이루어집니다.

## HTTP 특징

1. 비 연결 지향(connectionless)
   - 서버는 요청에 대한 응답을 보낸 후 연결을 끊음
   - 클라이언트는 서버와 서로 연결되어 있는 상태가 아님




<그림1_Django_Authentication System 01_HTTP 특징>

**TIP**

**HTTP가 비 연결 지향으로 설계된 이유는?**
- **사용자가 문서를 다 읽을 때까지 모든 사용자와의 연결을 계속 유지해야 한다면, 수많은 연결이 서버의 메모리와 자원을 차지하게 됩니다.**
- **이러한 자원 낭비를 막기 위해 HTTP는 비연결(Connectionless) 방식을 채택했습니다.**

## HTTP 특징

### 2. 무상태(stateless)

- 연결을 끊는 순간 클라이언트와 서버 간의 통신이 끝나며 상태 정보가 유지되지 않음
- 무상태의 의미
  - 장바구니에 담은 상품을 유지할 수 없음
  - 로그인 상태를 유지할 수 없음

**TIP**

**HTTP가 무상태(Stateless)로 설계된 이유는?**

- **서버가 모든 클라이언트 상태를 기억한다고 하면 저장하고, 관리해야 하므로 매우 복잡해집니다.**
- **더 큰 문제는 여러 대의 서버를 운영할 때, 클라이언트 상태를 공유하기 위해서 서로 다른 서버가 연결되어야 하는 문제로 인해서 확장성이 저하됩니다.**
- **결국 무상태는 서버의 부담을 없애고 어떤 서버든 자유롭게 요청을 처리할 수 있게 만들어, 대규모 웹 서비스를 구축하는 데 핵심적인 역할을 합니다.**

SAMSUNG SW AI ACADEMY FOR YOUTH

함께가요 미래로!
Enabling People

# 쿠키

## 쿠키

# 쿠키

**Cookie**

서버가 사용자의 웹 브라우저에
전송하는 작은 데이터 조각

쿠키는 웹사이트가 사용자의 브라우저에 남기는 작은 ‘데이터 조각’입니다

이를 통해 서버는 ‘나는 이전에 이 사이트에 방문 했었고, 로그인도 했어’ 와 같이 사용자를 기억하고 식별할 수 있습니다.

덕분에 매번 아이디와 비밀번호를 다시 입력할 필요 없이 자동 로그인이 유지되거나, 장바구니에 담은 상품이 저장되는 것입니다.

## 쿠키 특징

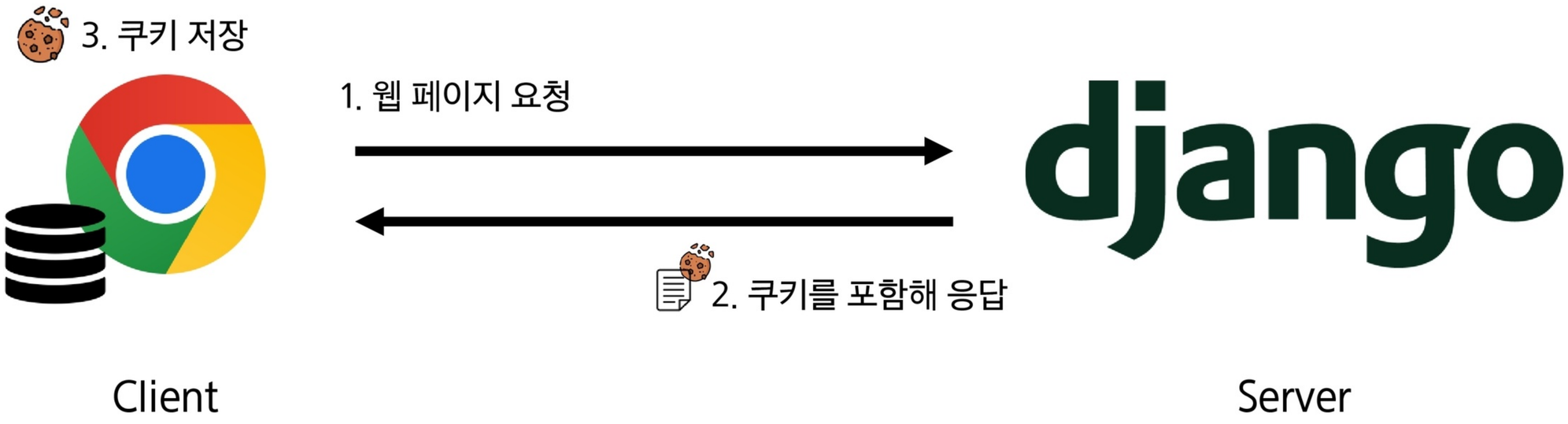
- 서버가 사용자의 웹 브라우저에 전송하는 작은 데이터 조각
- 사용자 인증, 추적, 상태 유지 등에 사용되는 데이터 저장 방식
- key-value 형식의 데이터

## 쿠키 사용 예시

- 로그인 유지(세션 관리)
- 장바구니
- 언어, 테마 등 사용자 설정 기억

## 쿠키 동작 예시 (1/2)

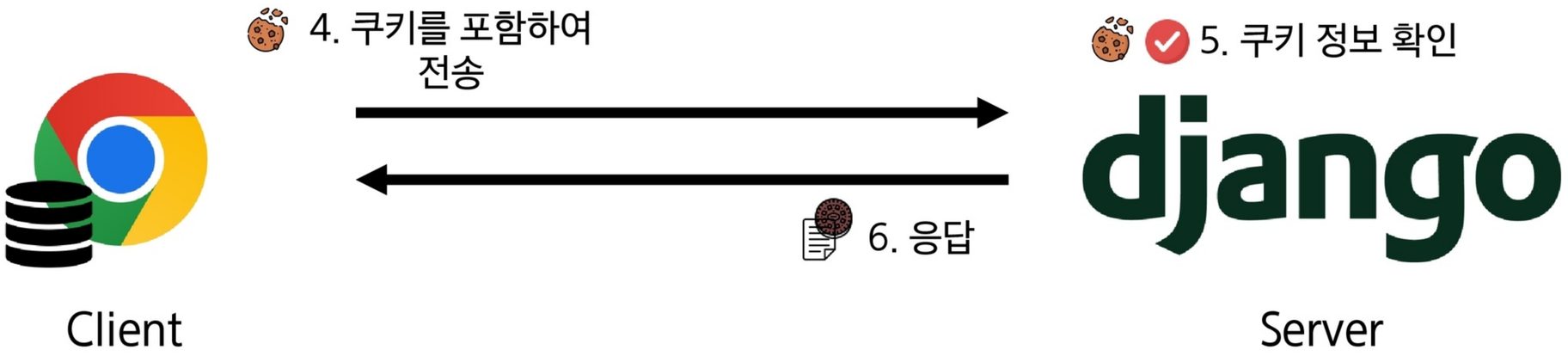
1. 브라우저가 웹 서버에 웹 페이지를 요청
2. 웹 서버는 요청된 페이지와 함께 쿠키를 포함한 응답을 브라우저에게 전송
3. 브라우저는 받은 쿠키를 저장소에 저장하고, 쿠키의 속성(만료 시간, 도메인, 주소 등)도 함께 저장됨




<그림2_Django_Authentication System 01_쿠키 동작 예시1>

## 쿠키 동작 예시 (2/2)

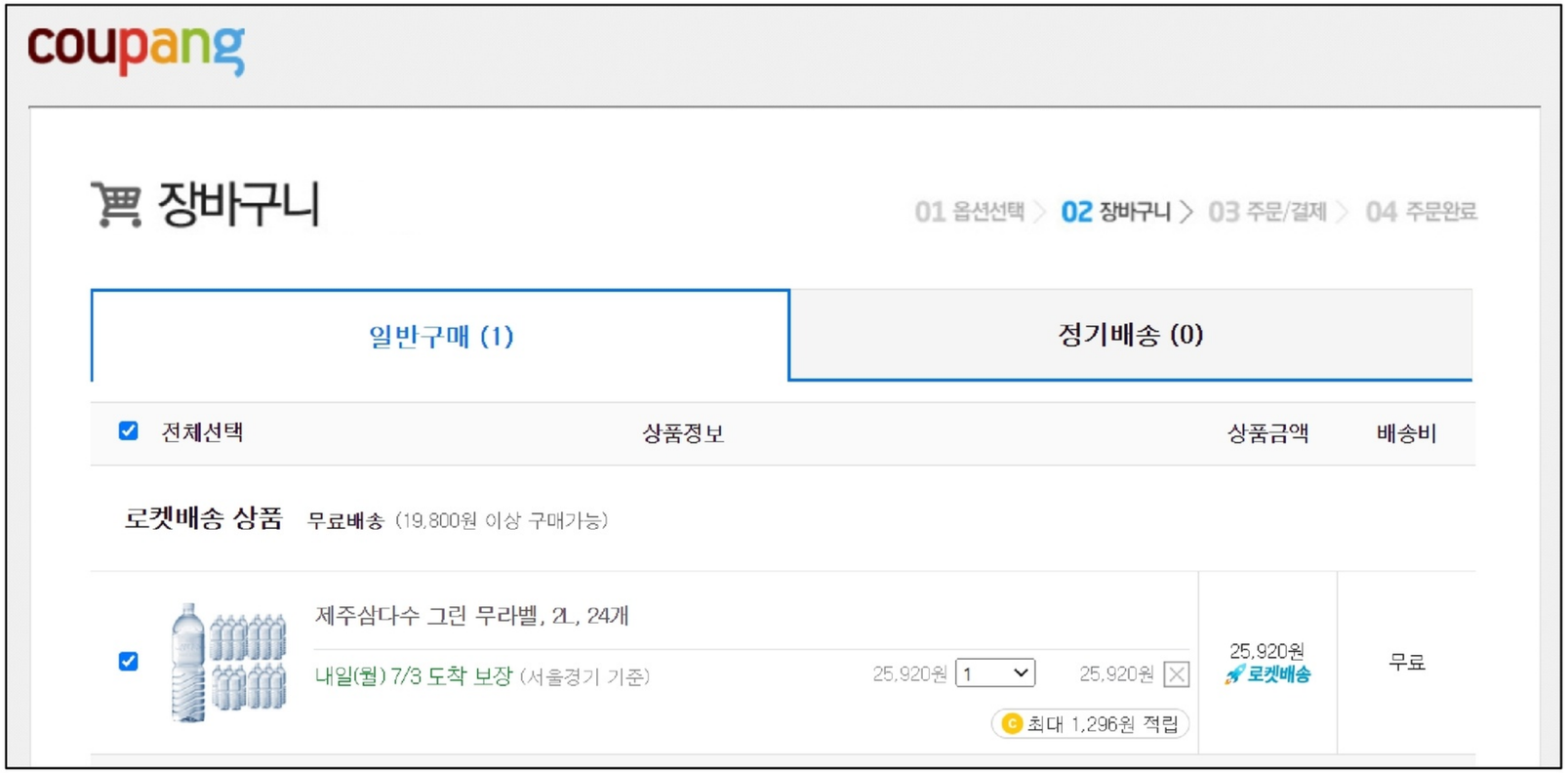
4. 이후 브라우저가 같은 웹 서버에 웹 페이지를 요청할 때, 저장된 쿠키 중 해당 요청에 적용 가능한 쿠키를 포함하여 함께 전송
5. 웹 서버는 받은 쿠키 정보를 확인하고, 필요에 따라 사용자 식별, 세션 관리 등을 수행
6. 웹 서버는 요청에 대한 응답을 보내며, 필요한 경우 새로운 쿠키를 설정하거나 기존 쿠키를 수정할 수 있음




<그림3_Django_Authentication System 01_쿠키 동작 예시2>

# 쿠키를 이용한 장바구니 예시 (1/5)

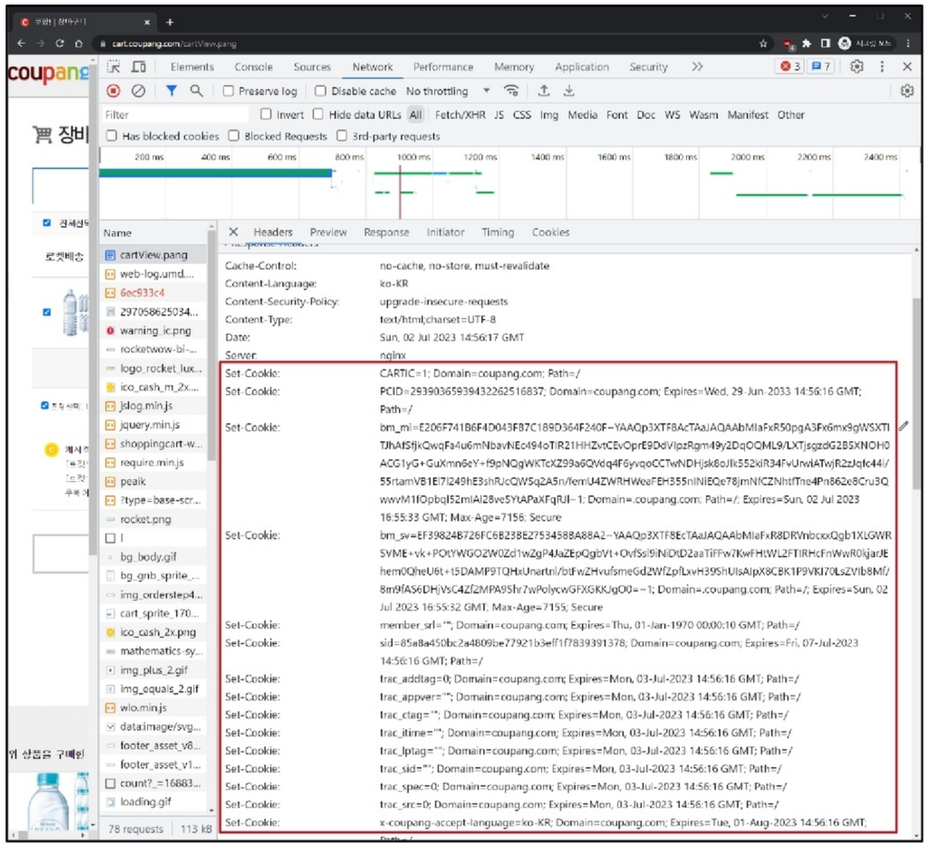
1. 장바구니에 상품 담기




<그림4_Django_Authentication System 01_쿠키 장바구니 예시1>

# 쿠키를 이용한 장바구니 예시 (2/5)

2. 서버는 응답과 함께 `Set-Cookie` 응답 헤더를 브라우저에게 전송
   - 개발자 도구 - Network 탭 - **`cartView.pang`** 확인
   - 이 헤더는 클라이언트에게 쿠키를 저장하라고 전달하는 것

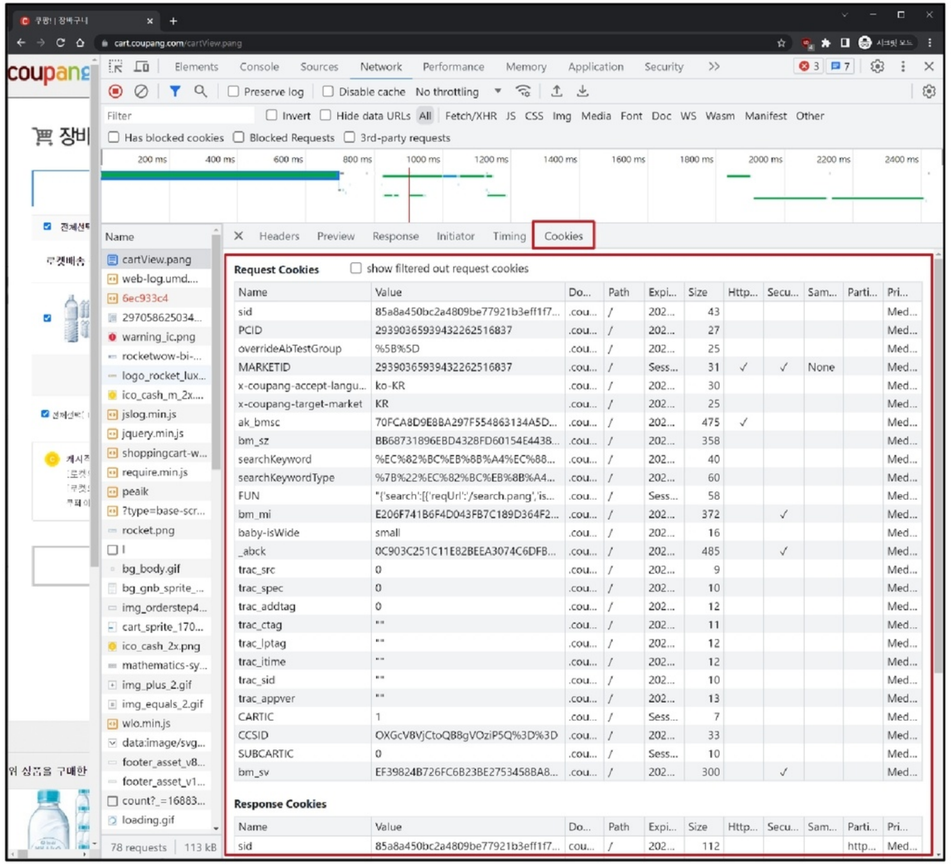



<그림5_Django_Authentication System 01_쿠키 장바구니 예시2>

# 쿠키를 이용한 장바구니 예시 (3/5)

3. Cookie 데이터 자세히 확인

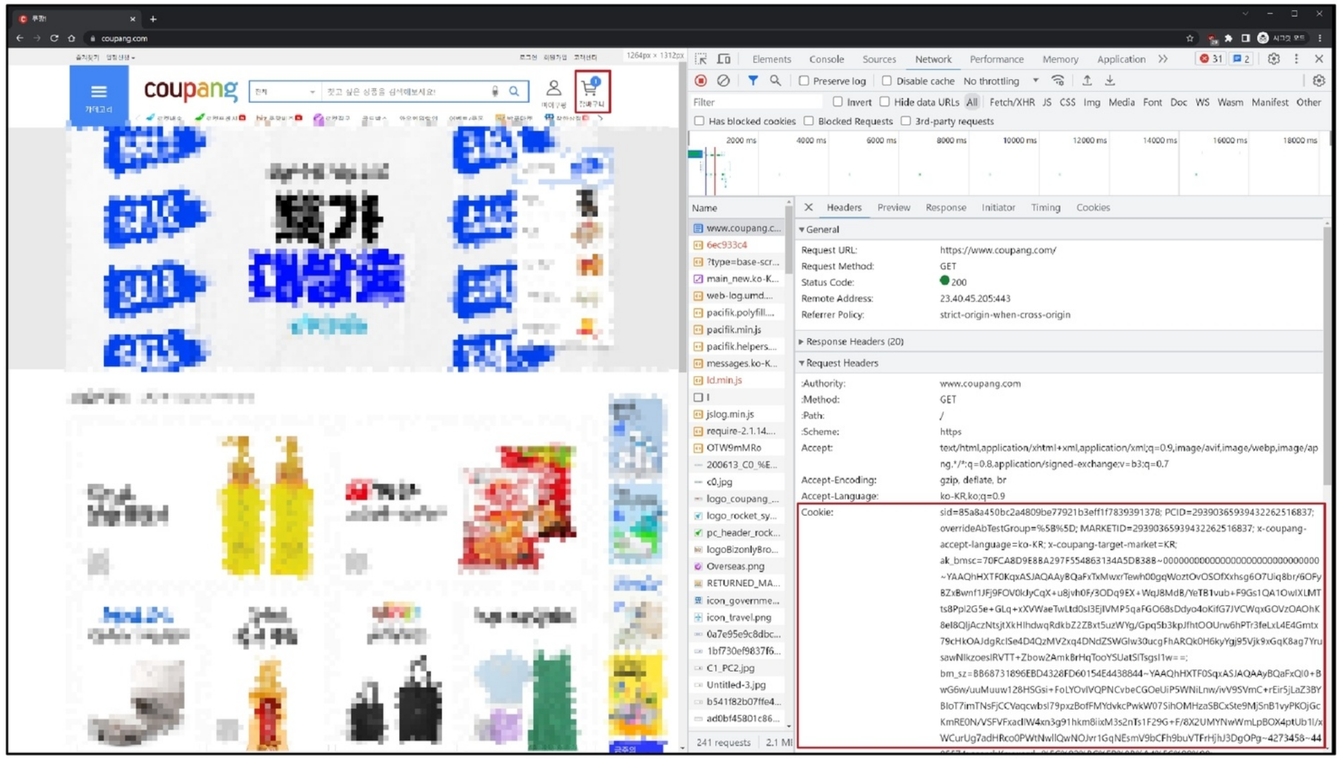
- `key – value` 형식의 데이터
- 단순히 값만 저장하는 것이 아니라, 여러 속성을 제어할 수 있음
- 하나의 웹사이트에서 많은 수의 쿠키를 동시에 사용
- `Request Cookies & Response Cookies` 탭을 보면
  쿠키의 양방향 통신 특징을 볼 수 있음




<그림6_Django_Authentication System 01_쿠키 장바구니 예시3>

# 쿠키를 이용한 장바구니 예시 (4/5)

4. 메인 페이지 이동 - 장바구니 유지 상태 확인

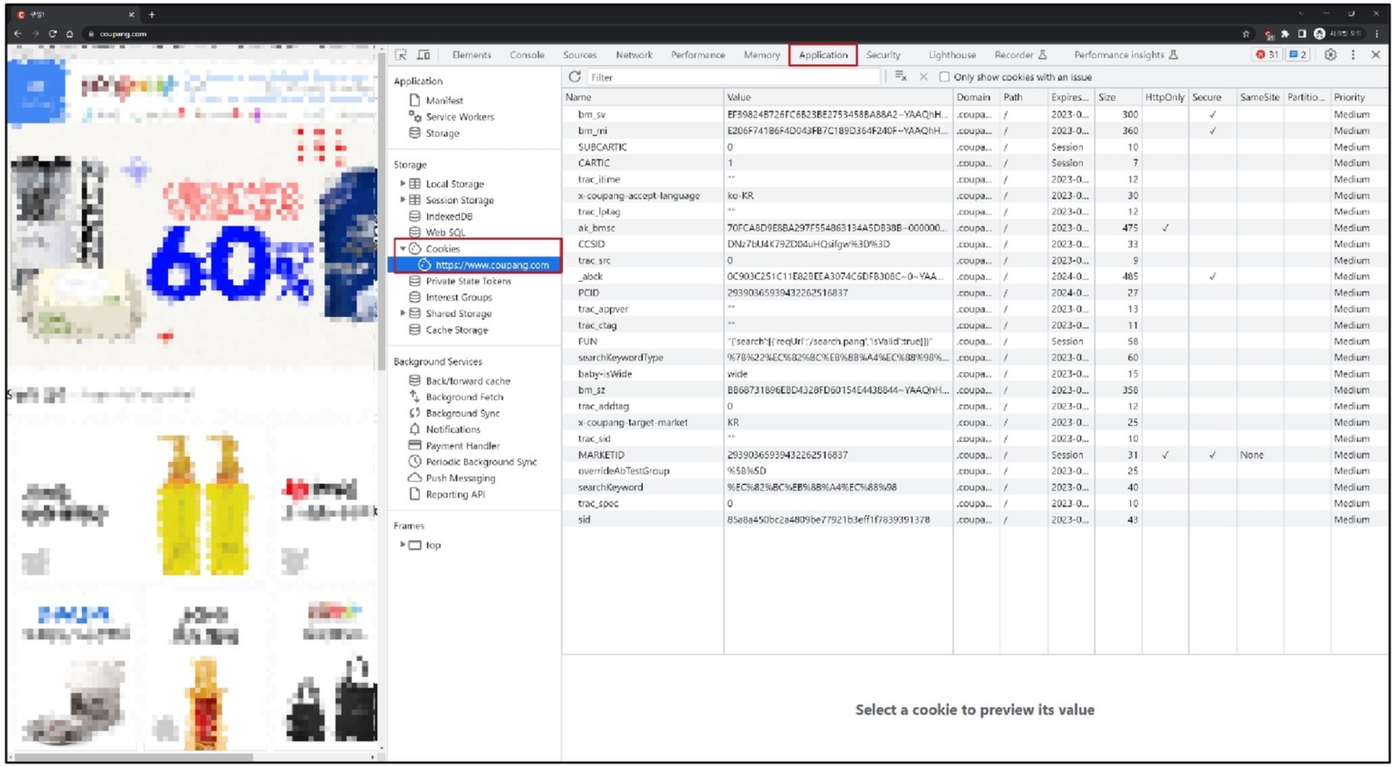



<그림7_Django_Authentication System 01_쿠키 장바구니 예시4>

# 쿠키를 이용한 장바구니 예시 (5/5)

5. 개발자 도구 - Application 탭 - Cookies

- 마우스 우측 버튼 - Clear - 새로고침 - 장바구니가 빈 것을 확인




<그림8_Django_Authentication System 01_쿠키 장바구니 예시5>

## 쿠키의 작동 원리와 활용 (1/2)

### 1. 쿠키 저장 방식

- 브라우저(클라이언트)는 쿠키를 KEY-VALUE의 데이터 형식으로 저장
- 쿠키에는 이름, 값 외에도 만료 시간, 도메인, 경로 등의 추가 속성이 포함 됨

### 2. 쿠키 전송 과정

- 서버는 HTTP 응답 헤더의 Set-Cookie 필드를 통해 클라이언트에게 쿠키를 전송
- 브라우저는 받은 쿠키를 저장해 두었다가, 동일한 서버에 재요청 시 HTTP 요청 Header의 Cookie 필드에 저장된 쿠키를 함께 전송

## 쿠키의 작동 원리와 활용 (2/2)

### 3. 쿠키의 주요 용도

- 두 요청이 동일한 브라우저에서 들어왔는지 아닌지를 판단할 때 주로 사용됨
- 이를 이용해 사용자의 로그인 상태를 유지할 수 있음
- 상태가 없는(stateless) HTTP 프로토콜에서 상태 정보를 기억시켜 주는 역할

➢ 서버에게 “나 로그인 된(인증된) 사용자야!” 라는 인증 정보가 담긴 쿠키를 매 요청마다 계속 보내는 것

## 쿠키 사용 목적

1. 세션 관리 (Session management)
   - 로그인, 아이디 자동완성, 공지 하루 안 보기, 팝업 체크, 장바구니 등의 정보 관리
2. 개인화 (Personalization)
   - 사용자 선호 설정(언어 설정, 테마 등) 저장
3. 추적, 수집 (Tracking)
   - 사용자 행동을 기록 및 분석

**TIP**

- **쿠키는 탈취될 수 있으니, 비밀번호 등 민감한 정보는 절대 직접 저장하면 안 됩니다.**
- **쿠키는 모든 요청에 포함되어 전송되므로, 크기를 최소화해야 사이트 성능에 유리합니다.**

SAMSUNG SW AI ACADEMY FOR YOUTH

함께가요 미래로!
Enabling People

# 세션

## 세션

# 세션

**session**

서버 측에서 생성되어
클라이언트와 서버 간의 상태를 유지,
상태 정보를 저장하는 데이터 저장 방식

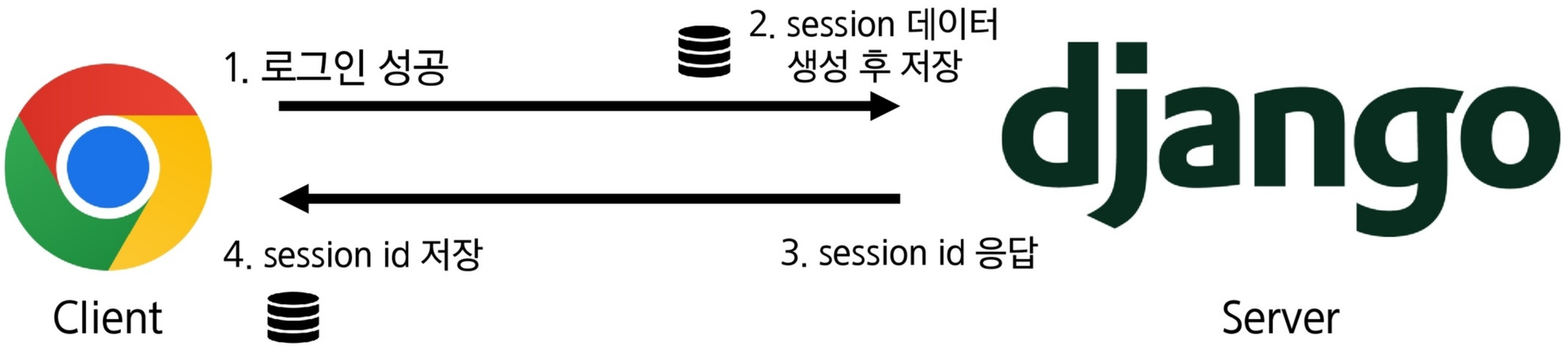
세션은 로그인 정보와 같은 중요 데이터를 클라이언트가 아닌 서버 쪽에 저장하고 유지하는 기술입니다.

서버는 각 사용자를 구분하기 위해 고유한 '세션 ID'를 발급하고, 이 ID만 쿠키에 담아 클라이언트에 보내 사용자를 식별합니다.

실제 데이터는 서버에만 보관되므로 쿠키만 사용하는 방식보다 훨씬 보안에 유리하며, 사용자의 로그인 상태를 안전하게 유지하는 데 주로 사용됩니다.

## 세션 작동 원리 (1/2)

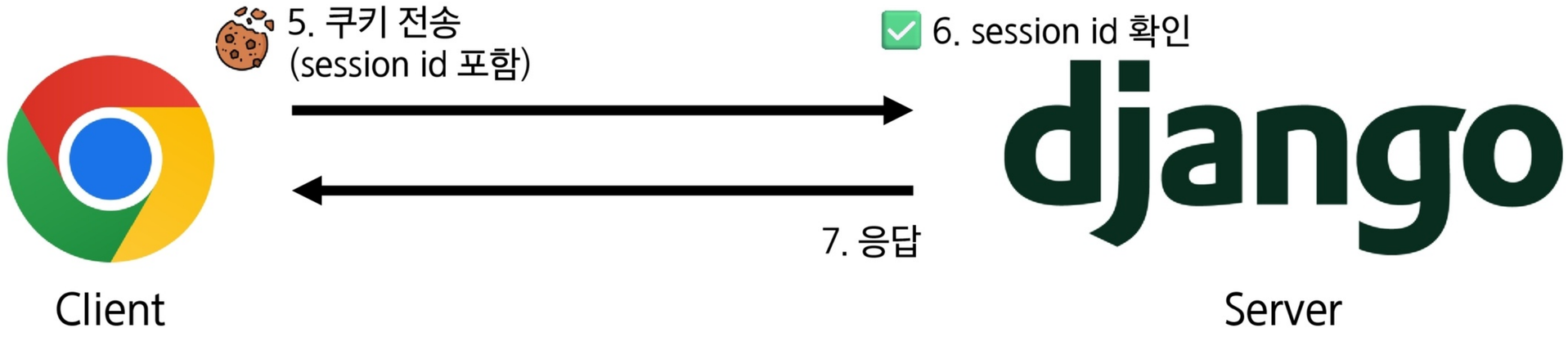
1. 클라이언트가 로그인 요청 후 인증에 성공하면 서버가 session 데이터를 생성 후 저장
2. 생성된 session 데이터에 인증 할 수 있는 session id를 발급
3. 발급한 session id를 클라이언트에게 응답 (데이터는 서버에 저장, 열쇠만 주는 것)
4. 클라이언트는 응답 받은 session id를 쿠키에 저장




<그림9_Django_Authentication System 01_세션 작동 원리1>

## 세션 작동 원리 (2/2)

5. 클라이언트가 다시 동일한 서버에 접속하면 요청과 함께 쿠키(session id가 저장된)를 서버에 전달
6. 쿠키는 요청 때마다 서버에 함께 전송 되므로 서버에서 session id를 확인해 로그인 되어있다는 것을 계속해서 확인하도록 함
7. 사용자의 요청을 처리하고 응답




<그림10_Django_Authentication System 01_세션 작동 원리2>

## 세션 특징

- 서버 측에서 생성되어 클라이언트와 서버 간의 상태를 유지
  - 서버의 메모리나 데이터베이스에 저장되므로, 서버 리소스를 사용(효율적 관리 필요)
- 상태 정보를 저장하는 데이터 저장 방식
- 쿠키에 세션 ID를 저장하여 매 요청마다 세션 ID를 함께 보냄
- 세션은 영구적으로 유지되지 않음

※ 중요한 데이터를 저장하므로 보안을 신경써야 함

※ 공격자가 세션 ID를 탈취하면, 해당 사용자인 것처럼 위장하여 서버에 접근이 가능하므로 유의

## 세션 정리

1. 서버 측에서는 세션 데이터를 생성 후 저장하고, 이 데이터에 접근할 수 있는 세션 ID를 생성
2. 이 ID를 클라이언트 측으로 전달하고, 클라이언트는 쿠키에 이 ID를 저장
3. 이후 클라이언트가 같은 서버에 재요청 시마다 저장해 두었던 쿠키도 요청과 함께 전송
   - 로그인 상태 유지를 위해 로그인 되어있다는 사실을 입증하는 데이터를 매 요청마다 계속해서 보내는 것

## 쿠키와 세션의 목적

- 클라이언트와 서버 간의 상태 정보를 유지하고 사용자를 식별하기 위해 사용

# Django Authentication System

## 인증의 필요성

- 클라이언트와 서버 간의 상태 정보를 유지하기 위해서 쿠키와 세션을 사용
- 서버는 수많은 요청이 각각 누구에게서 왔는지 구별해야 함
- 그래서 사용자를 식별하기 위해서 필요한 과정이 바로 '인증(Authentication)'
- 다양한 인증이 존재
  - 아이디와 비밀번호
  - 소셜 로그인(OAuth)
  - 생체인증

➢ Django 에서는 사용자 인증과 관련된 가장 중요하고 기본적인 뼈대를 제공 (Django Authentication System)

## Django Authentication System

- Django에서 사용자 인증과 관련된 기능을 모아 놓은 시스템
- 인증에 중요한 기본적인 기능을 제공
  - **User Model**: 사용자 인증 후 연결될 User Model 관리
  - **Session 관리**: 로그인 상태를 유지하고 서버에 저장하는 방식을 관리
  - **기본 인증(Id/Password)**: 로그인/로그아웃 등 다양한 기능을 제공

➢ Django Authentication System을 활용하여 로그인/로그아웃/회원가입/회원정보수정 등 다양한 기능들을 구현해보자!

## 기본 User Model의 한계

- 인증 후 사용되는 User Model은 별도의 User 클래스 정의 없이 내장된 auth 앱에 작성된 User 클래스를 사용했음
- 하지만 Django의 기본 User 모델은 username, password 등 제공되는 필드가 매우 제한적
- 추가적인 사용자 정보(예: 생년월일, 주소, 나이 등)가 필요하다면 이를 위해 기본 User Model을 변경하기 어려움
  - 별도의 설정 없이 사용할 수 있어 간편하지만, 개발자가 직접 수정하기 어려움

https://github.com/django/django/blob/main/django/contrib/auth/models.py#L446

## 내장된 auth 앱

- 기본적으로 username, password, email 등의 필드를 가진 User 모델을 제공
- 단순히 로그인 여부만 확인하는 것을 넘어, 사용자별 또는 그룹별로 특정 행동에 대한 권한 부여가 가능

```
# settings.py
INSTALLED_APPS = [
    'django.contrib.admin',
    'django.contrib.auth',
    'django.contrib.contenttypes',
    'django.contrib.sessions',
    'django.contrib.messages',
    'django.contrib.staticfiles',
]
```

## User Model 대체의 필요성

- 프로젝트의 특정 요구사항에 맞춰 사용자 모델을 확장할 수 있음
- 예를 들어
  - 이메일을 `username`으로 사용하거나, 다른 추가 필드를 포함시킬 수 있음
  - 기본 User 모델의 `first_name, last_name`처럼 우리 프로젝트에 필요 없는 필드를 제거하여 데이터베이스 모델을 더 간결하게 관리할 수 있음

➢ Django에서 제공하는 기본 유저 모델이 아닌 우리가 직접 커스텀한 유저 모델을 사용해보자!

SAMSUNG SW AI ACADEMY FOR YOUTH

함께가요 미래로!
Enabling People

# Custom User model

SAMSUNG SW AI ACADEMY FOR YOUTH

함께가요 미래로!
Enabling People

# User model 대체하기

## 사전 준비

- 두 번째 app accounts 생성 및 등록
- auth와 관련한 경로나 키워드들을 django 내부적으로 accounts라는 이름으로 사용하고 있기 때문에 되도록 ‘accounts’로 지정하는 것을 권장
  - 예: 로그인이 필요한 페이지에 접근하면 Django는 기본적으로 `/accounts/login/`으로 보냄(`LOGIN_URL` 기본값)

```python
# accounts/urls.py

from django.urls import path
from . import views

app_name = 'accounts'
urlpatterns = []
```

```python
# crud/urls.py

urlpatterns = [
    ... ,
    path('accounts/', include('accounts.urls')),
]
```

## Custom User Model로 대체하기 (1/3)

- AbstractUser 클래스를 상속받는 커스텀 User 클래스 작성

➢ 기존 User 클래스도 AbstractUser를 상속받기 때문에 커스텀 User 클래스도 기존 User 클래스와 완전히 같은 모습을 가지게 됨

```
# accounts/models.py

from django.contrib.auth.models import AbstractUser

class User(AbstractUser):
    pass
```

https://github.com/django/django/blob/main/django/contrib/auth/models.py#L517

## Custom User Model로 대체하기 (2/3)

- django 프로젝트에서 사용하는 기본 User 모델을 우리가 작성한 User 모델로 사용할 수 있도록 AUTH_USER_MODEL 값을 변경
  - 수정 전 기본 값은 'auth.User'
- accounts 앱에 작성한 User 모델을 기본 모델로 설정

```python
# accounts/models.py

from django.contrib.auth.models import AbstractUser

class User(AbstractUser):
    pass
```

```python
# settings.py

AUTH_USER_MODEL = 'accounts.User'
```

## Custom User Model로 대체하기 (3/3)

- admin site에 대체한 User 모델 등록
  - 기본 User 모델이 아니기 때문에 등록하지 않으면 admin 페이지에 출력되지 않기 때문

```
# accounts/admin.py

from django.contrib import admin
from django.contrib.auth.admin import UserAdmin
from .models import User


admin.site.register(User, UserAdmin)
```

## AUTH_USER_MODEL

# AUTH_USER_MODEL

Django 프로젝트의 User를 나타내는 데 사용하는 모델을 지정하는 속성

**※ 주의 ※**

**Django**는 프로젝트 중간에 AUTH_USER_MODEL을 변경하는 것을
강력하게 권장하지 않음

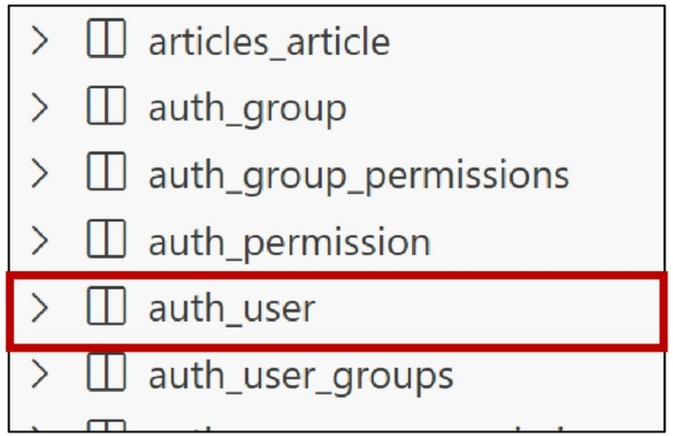
※ 이미 프로젝트가 진행되고 있을 경우 데이터베이스 초기화 후 진행

https://docs.djangoproject.com/en/5.2/ref/settings/#auth-user-model

## 사용하는 User 테이블의 변화

- accounts에 만든 커스텀 유저 모델이 데이터베이스에 반영 됨

1. 기존 `db.sqlite3` 삭제, 각 앱 migrations 폴더의 번호 파일 삭제 (`__init__.py`는 남김)
2. `python manage.py makemigrations`
3. `python manage.py migrate`




<그림11_Django_Authentication System 01_User 테이블 변화 전>

<그림12_Django_Authentication System 01_User 테이블 변화 후>

## 프로젝트를 시작하며 반드시 User 모델을 대체해야 함

- Django는 새 프로젝트를 시작하는 경우 비록 기본 User 모델이 충분하더라도 커스텀 User 모델을 설정하는 것을 강력하게 권장하고 있음
- 커스텀 User 모델은 기본 User 모델과 동일하게 작동(1) 하면서도 나중에 맞춤 설정(2)할 수 있음

※ 단, User 모델 대체 작업은 프로젝트의 모든 migrations 혹은 첫 migrate를 실행하기 전에 이 작업을 마쳐야 함

**TIP**

- 지금 당장 필요 없어도 만드세요. 나중에 닉네임 등 필드를 추가하기가 매우 쉬워집니다.
- 모델 생성 후, `settings.py`의 `AUTH_USER_MODEL`을 설정해야만 Django가 이 모델을 사용합니다.

# Login

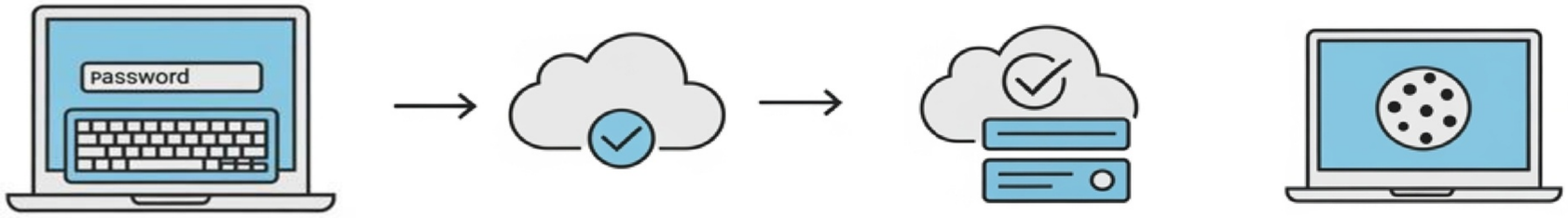
## Login

- 클라이언트와 서버 간의 상태 정보를 유지하기 위해서 쿠키와 세션을 사용
- 서버는 수많은 요청이 각각 누구에게서 왔는지 구별해야 함
- 서버에 "나"임을 인증하는 과정이 바로 "Login"

**Session**
서버 측에서 생성되어 클라이언트와 서버 간의 상태를 유지, 상태 정보를 저장하는 데이터 저장 방식

<개념1_Django_Authentication System 01_세션>

➢결국 Login은 인증(id/password)을 완료한 뒤, 서버에 Session을 생성(Create)하고, 그 Session을 찾을 수 있는 Session ID를 쿠키에 담아 클라이언트에게 전달하는 것




1. 로그인(id/password)
2. 로그인 인증
3. 세션 생성 후 서버 저장
4. 서버가 세션을 만들어 클라이언트와 연결하는 과정

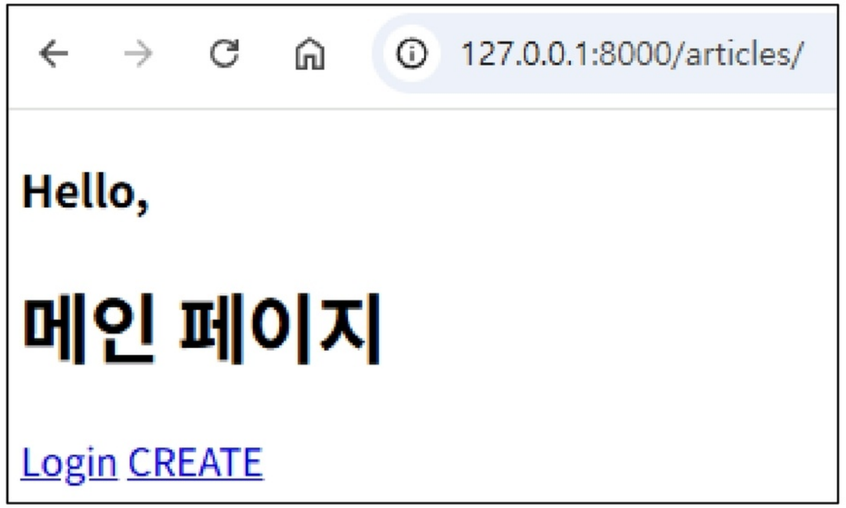
<그림13_Django_Authentication System 01_로그인 과정>

## 로그인 기능 구현 (1/4)

- 로그인 경로 url 및 링크 생성

```
# accounts/urls.py

app_name = 'accounts'
urlpatterns = [
    path('login/', views.login, name='login'),
]
```

```
<!-- articles/index.html -->

<h1>메인 페이지</h1>
<a href="{% url "accounts:login" %}">Login</a>
```




<그림13_Django_Authentication System 01_로그인 화면>

## 로그인 기능 구현 (2/4)

- .../accounts/login/ url로 요청이 들어왔을 때 실행할 login 함수 작성
- 로그인 인증에 사용할 데이터를 입력 받는 built-in form (AuthenticationForm) 사용

```python
# accounts/views.py

from django.contrib.auth.forms import AuthenticationForm

def login(request):
    if request.method == 'POST':
        pass
    else:
        form = AuthenticationForm()
    context = {
        'form': form,
    }
    return render(request, 'accounts/login.html', context)
```

**TIP**

로그인 view가 해야 하는 일은 2가지
1. 로그인 입력 페이지 제공 => 페이지를 조회하는 요청이므로 GET
2. 사용자 정보를 받아 실제 로그인(세션 생성) => 세션을 만드는 요청이므로 POST

➢ 게시글 생성(create) view와 똑같습니다. 결국 로그인은 세션을 생성(Create)하는 기능입니다.

## 게시글 생성과 로그인 비교

| 구분 | 게시글 create view | 로그인 login view |
|---|---|---|
| GET 요청 | 게시글 작성 페이지 제공 | 로그인 페이지 제공 |
| POST 요청 | Article 생성 (DB: articles_article) | Session 생성 (DB: `django_session`) |
| 사용하는 Form | ArticleForm (ModelForm) | AuthenticationForm (Form) |
| 성공 시 | `form.save()` | `auth_login(request, form.get_user())` |

## AuthenticationForm()

- 로그인 인증에 사용할 데이터를 입력 받는 built-in form
- User 모델과 직접 연결된 “ModelForm”이 아닌, 일반 “Form”을 상속 받음
  - 일반 “Form”이기 때문에 사용자를 생성하거나 수정하는 용도가 아닌 “인증”하는 역할만 수행
  - `ArticleForm`과 부모 클래스가 다르기 때문에, 생성자 함수의 인자 구성도 다를 수 있다는 점을 예상해야 함 (슬라이드 53, 54에서 확인)

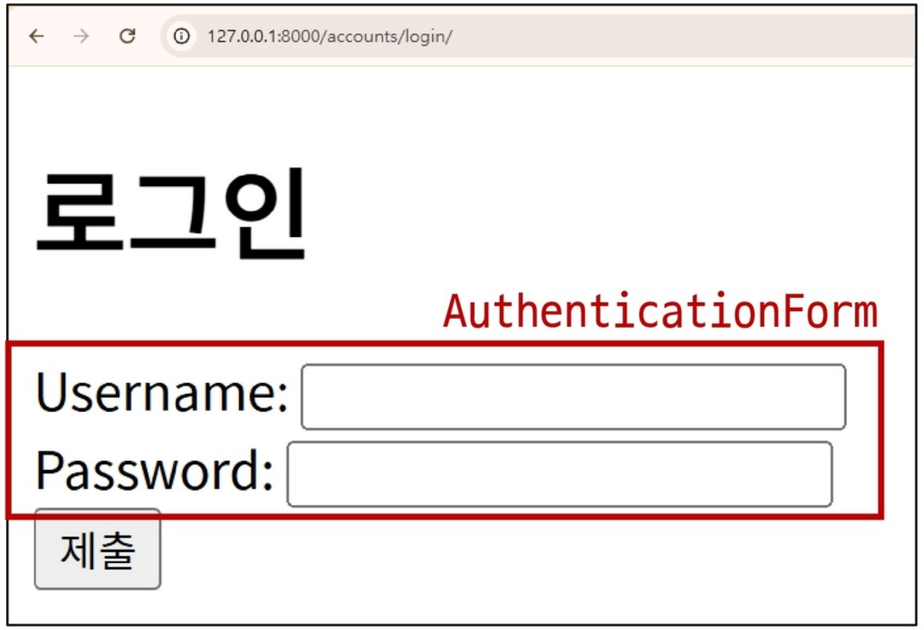
```
class AuthenticationForm(forms.Form):
    """
    Base class for authenticating users. Extend this to get a form that accepts
    username/password logins.
    """

    username = UsernameField(widget=forms.TextInput(attrs={"autofocus": True}))
    password = forms.CharField(
        label=_("Password"),
        strip=False,
        widget=forms.PasswordInput(attrs={"autocomplete": "current-password"}),
    )
```

<그림14_Django_Authentication System 01_AuthenticationForm>

## 로그인 기능 구현 (3/4)

- 로그인은 세션을 생성하는(서버 상태를 바꾸는) 요청이므로 POST 방식을 사용 (비밀번호가 URL이나 브라우저 기록에 남지 않는 효과도 있음)
- CSRF 공격을 방지하기 위해 `csrf_token` 작성
- 서버로부터 전달받은 `AuthenticationForm`을 화면에 출력

```
<!-- accounts/login.html -->

<h1>로그인</h1>
<form action="{% url 'accounts:login' %}" method="POST">
  {% csrf_token %}
  {{ form }}
  <input type="submit">
</form>
```




<그림15_Django_Authentication System 01_로그인 화면>

## 로그인 기능 구현 (4/4)

- 사용자가 입력한 로그인 정보를 입력 받는 로직을 “POST” 조건문에 작성
- 입력 받은 정보를 기반으로 로그인하여 세션을 만드는 “login” 함수를 활용

```python
# accounts/views.py
from django.shortcuts import render, redirect
from django.contrib.auth import login as auth_login

def login(request):
    if request.method == 'POST':
        form = AuthenticationForm(request, request.POST)
        # form = AuthenticationForm(request, data=request.POST)
        if form.is_valid():
            user = form.get_user()
            auth_login(request, user)
            return redirect('articles:index')
    else:
        form = AuthenticationForm()
    context = {
        'form': form,
    }
    return render(request, 'accounts/login.html', context)
```

```python
if form.is_valid():
    # 유효성 검사를 통과하면 인증된 유저 객체를 반환
    user = form.get_user()
    # 세션 생성
    auth_login(request, user)
    return redirect('articles:index')
```

## AuthenticationForm의 첫 번째 인자가 request인 이유

```
form = AuthenticationForm(request, request.POST)
```

- `ArticleForm(request.POST)` ➔ `ModelForm`은 첫 번째 인자가 `data`
- `AuthenticationForm(request, request.POST)` ➔ `AuthenticationForm`은 첫 번째 인자가 `request`, 두 번째가 `data`

➢ `AuthenticationForm(request.POST)`로 쓰면 에러 없이 로그인만 안 됩니다.
`request.POST`가 `request` 자리에 들어가고 `data`는 비어 있게 되어, `is_valid()`가 조용히 `False`가 되기 때문입니다.

## login(request, user)

- AuthenticationForm을 통해 인증된 사용자를 로그인 하는 함수
- request
  - 현재 사용자의 세션 정보에 접근하기 위해 사용
- user
  - 어떤 사용자가 로그인 되었는지를 기록하기 위해 사용

## get_user()

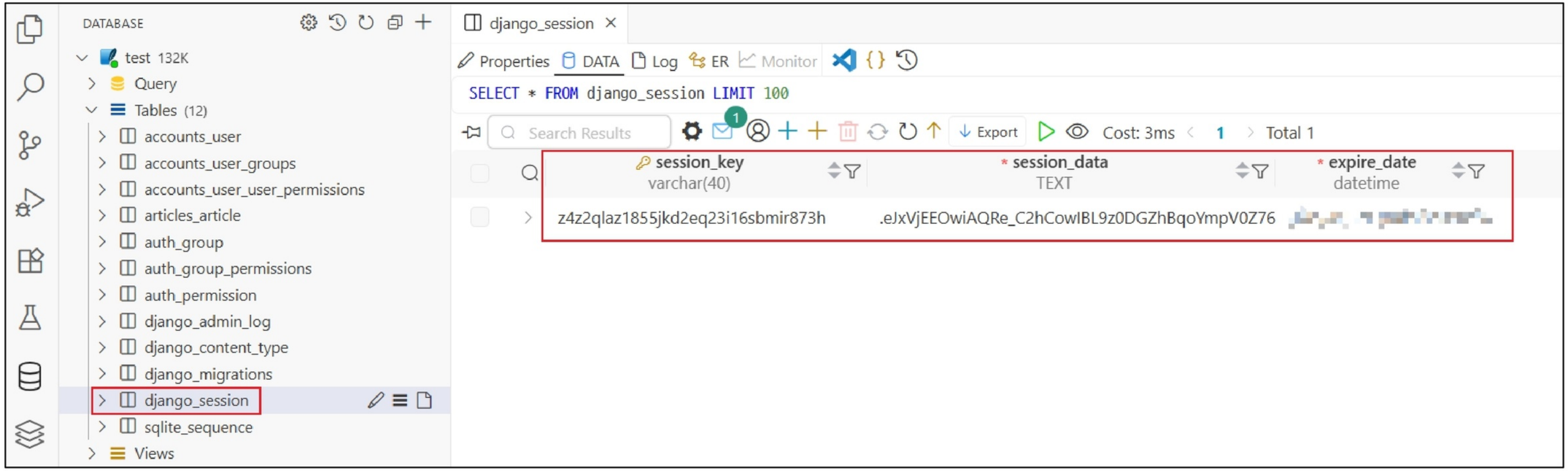
- AuthenticationForm의 인스턴스 메서드

```python
# accounts/views.py

from django.shortcuts import render, redirect
from django.contrib.auth import login as auth_login

def login(request):
    if request.method == 'POST':
        form = AuthenticationForm(request, request.POST)
        # form = AuthenticationForm(request, data=request.POST)
        if form.is_valid():
            user = form.get_user()
            auth_login(request, user)
            return redirect('articles:index')
    else:
        form = AuthenticationForm()
    context = {
        'form': form,
    }
    return render(request, 'accounts/login.html', context)
```

➢ 유효성 검사를 통과했을 경우, 로그인 한 사용자 객체를 반환

# 세션 데이터 확인하기 (1/2)

1. 로그인 후 발급받은 세션 확인
   - `django_session` 테이블에서 확인

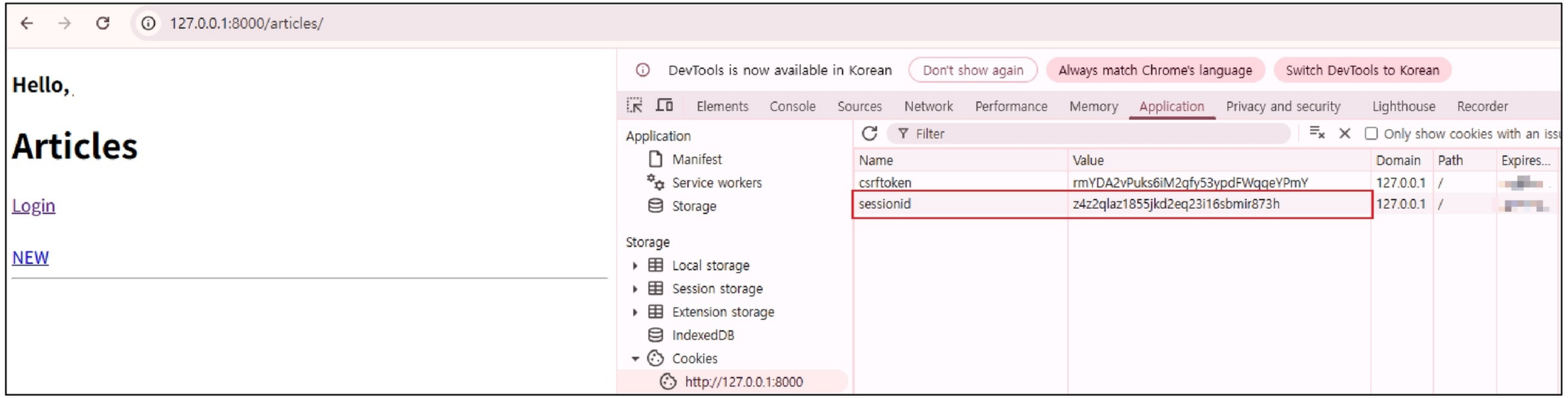



<그림16_Django_Authentication System 01_세션 DB 확인>

# 세션 데이터 확인하기 (2/2)

## 2. 브라우저에서 확인

- 개발자도구 - Application - Cookies




<그림17_Django_Authentication System 01_세션 개발자 도구 확인>

SAMSUNG SW AI ACADEMY FOR YOUTH

함께가요 미래로!
Enabling People

# Template with Authentication data

## Template with Authentication data

- 템플릿에서 인증 관련 데이터를 출력하는 방법

## 현재 로그인 되어있는 유저 정보 출력하기

- user라는 context 데이터를 사용할 수 있는 이유는?

➢ django가 미리 준비한 context 데이터가 존재하기 때문(context processors)

```
<!-- articles/index.html -->

<h3>Hello, {{ user.username }}</h3>
```

<그림18_Django_Authentication System 01_메인 페이지 로그인 링크>

## context processors

- 템플릿이 렌더링 될 때 호출 가능한 컨텍스트 데이터 목록
- 작성된 컨텍스트 데이터는 기본적으로 템플릿에서 사용 가능한 변수로 포함됨
- django에서 자주 사용하는 데이터 목록을 미리 템플릿에 로드 해 둔 것

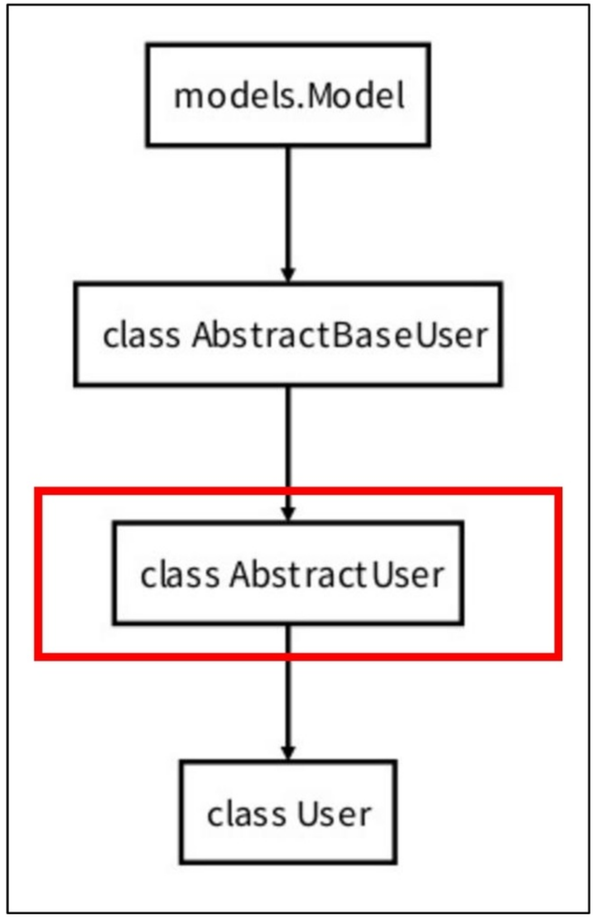
```
# settings.py

TEMPLATES = [
    {
        …,
        'OPTIONS': {
            'context_processors': [
                'django.template.context_processors.request',
                'django.contrib.auth.context_processors.auth',
                'django.contrib.messages.context_processors.messages',
            ],
        },
    },
]
```

# 참고

# AbstractUser class

## Abstract base classes (추상 기본 클래스)

- 몇 가지 공통 정보를 여러 다른 모델에 넣을 때 사용하는 클래스
- 데이터베이스 테이블을 만드는 데 사용되지 않으며, 대신 다른 모델의 기본 클래스로 사용되는 경우 해당 필드가 하위 클래스의 필드에 추가 됨




<그림8_Django_Authentication System 02_추상 클래스 관계>

## AbstractUser class

- 관리자 권한과 함께 완전한 기능을 가지고 있는 User model을
- 구현하는 추상 기본클래스
- 기본 User 모델이 가진 모든 필드가 이미 구현되어 있음

## 추상 기본 클래스(Abstract Base Class) 정리

| 구분 | AbstractBaseUser | AbstractUser |
|---|---|---|
| 제공 필드 | **최소한의 인증 필드** (비밀번호, last_login 등) | **기본 User 모델의 모든 필드** (username, email 등) |
| 장점 | 최대의 유연성과 자유도 | 개발 속도가 빠르고 편리함 |
| 사용 케이스 | 아이디(username) 대신 이메일이나 전화번호로 로그인하는 등, **완전히 새로운 인증 체계**를 만들 때 | 기존 인증 방식은 유지하면서 **프로필 사진, 닉네임 등 필드만 추가**하고 싶을 때 (대부분의 경우) |
| 예시 | 기본 피자 도우 | 토핑이 올려진 피자 |

<표1_Django_Authentication System 02_추상 기본 클래스 정리>

SAMSUNG SW AI ACADEMY FOR YOUTH

함께가요 미래로!
Enabling People

# 쿠키의 수명

# 쿠키 종류별 Lifetime (수명)

## 1. Session cookie

- 현재 세션(current session)이 종료되면 삭제됨
- 브라우저 종료와 함께 쿠키가 삭제됨 (서버의 세션 데이터가 지워지는 것은 아님)

## 2. Persistent cookies

- Expires 속성에 지정된 날짜 혹은 Max-Age 속성에 지정된 기간이 지나면 삭제됨

<그림19_Django_Authentication System 01_쿠키 종류별 수명>

SAMSUNG SW AI ACADEMY FOR YOUTH

함께가요 미래로!
Enabling People

# 쿠키와 보안

## 쿠키의 보안 장치

### 1. 제한된 정보

- 쿠키에는 비밀번호 같은 민감 정보 대신, 그 자체로는 의미 없는 값(세션 ID 등)만 저장.
  단, 세션 ID도 탈취되면 위험하므로 Django는 sessionid 쿠키에 HttpOnly(자바스크립트 접근 차단) 속성을 기본으로 적용

### 2. 암호화

- 중요한 정보는 서버에서 암호화해서 쿠키에 저장

### 3. 만료 시간

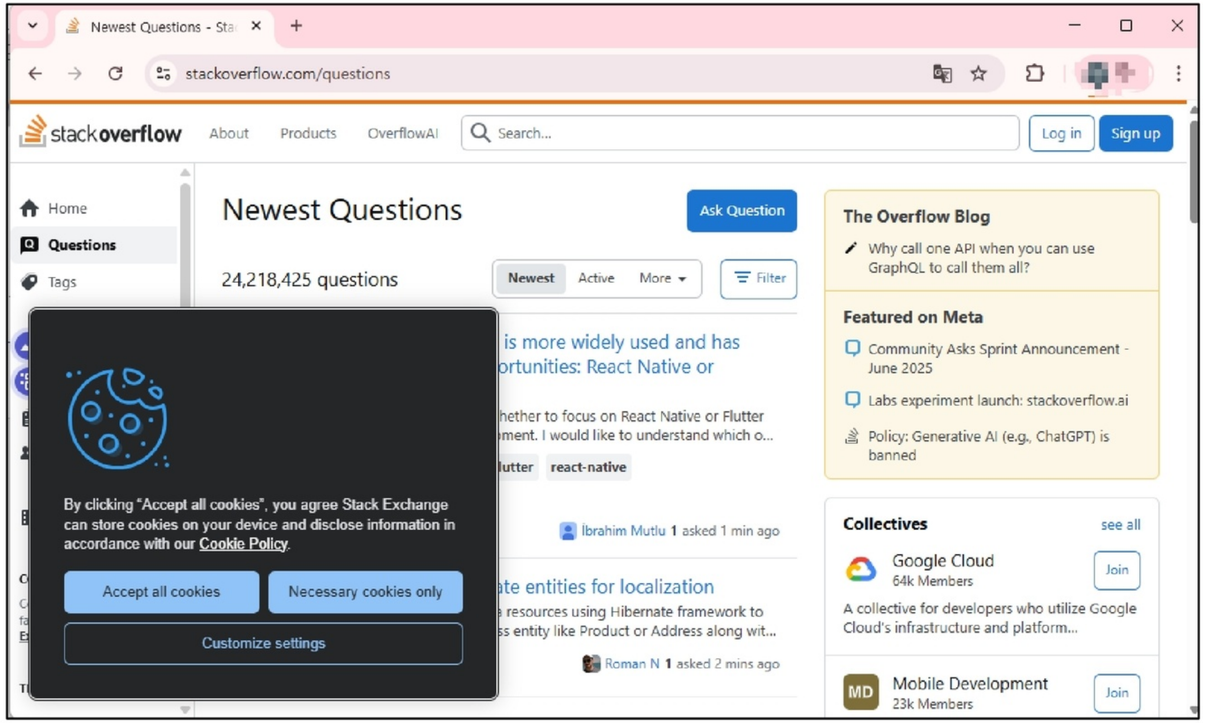
- 쿠키에는 만료 시간을 설정, 시간이 지나면 자동으로 삭제

### 4. 도메인 제한

- 쿠키는 특정 웹사이트에서만 사용할 수 있도록 설정할 수 있음

## 쿠키와 개인정보 보호

- 많은 국가에서 쿠키 사용에 대한 사용자 동의를 요구하는 법규를 시행
- 웹사이트는 쿠키 정책을 명시하고, 필요한 경우 사용자의 동의를 얻어야 함




<그림20_Django_Authentication System 01_쿠키 동의화면>

SAMSUNG SW AI ACADEMY FOR YOUTH

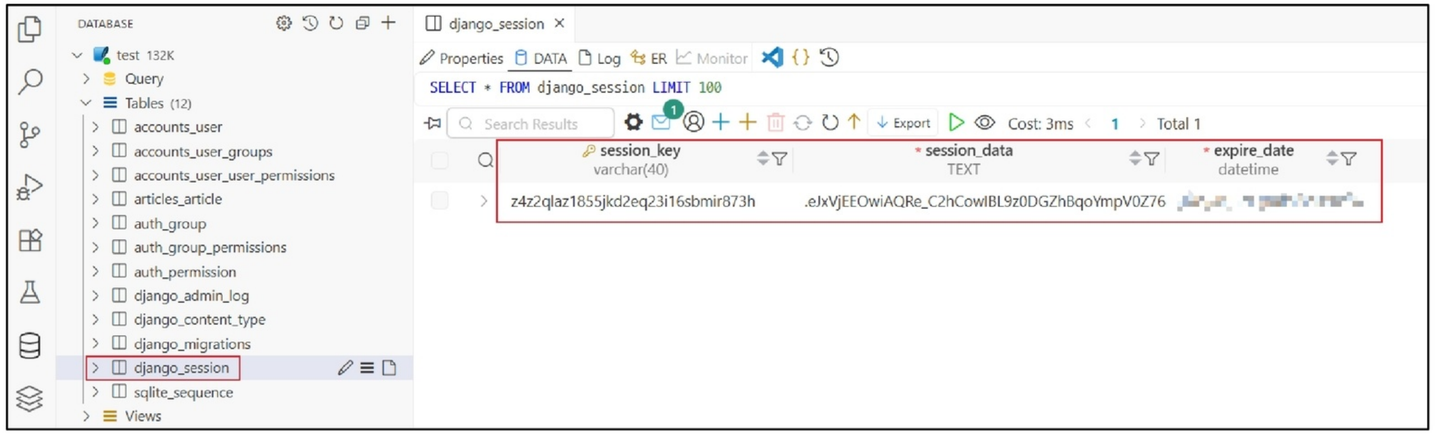
함께가요 미래로!
Enabling People

# Django에서의 세션 관리

## 세션 in Django

- Django는 'database-backed sessions' 저장 방식을 기본 값으로 사용
- session 정보는 DB의 **django_session** 테이블에 저장
- Django는 요청안에 특정 session id를 포함하는 쿠키를 사용해서 각각의 브라우저와 사이트가 연결된 session 데이터를 알아냄

➢ Django는 우리가 session 메커니즘(복잡한 동작원리)에 대부분을 생각하지 않게끔 많은 도움을 줌




<그림16_Django_Authentication System 01_세션 DB 확인>

SAMSUNG SW AI ACADEMY FOR YOUTH

함께가요 미래로!
Enabling People

# AuthenticationForm 내부 코드

## django github 코드 참고

- AuthenticationForm()
  - https://github.com/django/django/blob/main/django/contrib/auth/forms.py#L302
- AuthenticationForm()의 get_user() 인스턴스 메서드
  - https://github.com/django/django/blob/main/django/contrib/auth/forms.py#L373

```python
class AuthenticationForm(forms.Form):
    """
    Base class for authenticating users. Extend this t
    username/password logins.
    """

    username = UsernameField(widget=forms.TextInput(at
    password = forms.CharField(
        label=_("Password"),
        strip=False,
        widget=forms.PasswordInput(attrs={"autocomplete": "current-password"}),
    )

    error_messages = {
```

```python
def __init__(self, request=None, *args, **kwargs):
    """
    The 'request' parameter is set for custom auth use by subclasses.
    The form data comes in via the standard 'data' kwarg.
    """
    self.request = request
    self.user_cache = None
    super().__init__(*args, **kwargs)

    # Set the max length and label for the "username" field
```

<그림15_Django_Authentication System 01_AuthenticationForm>

```python
def get_user(self):
    return self.user_cache
```

<그림21_Django_Authentication System 01_AuthenticationForm getuser>

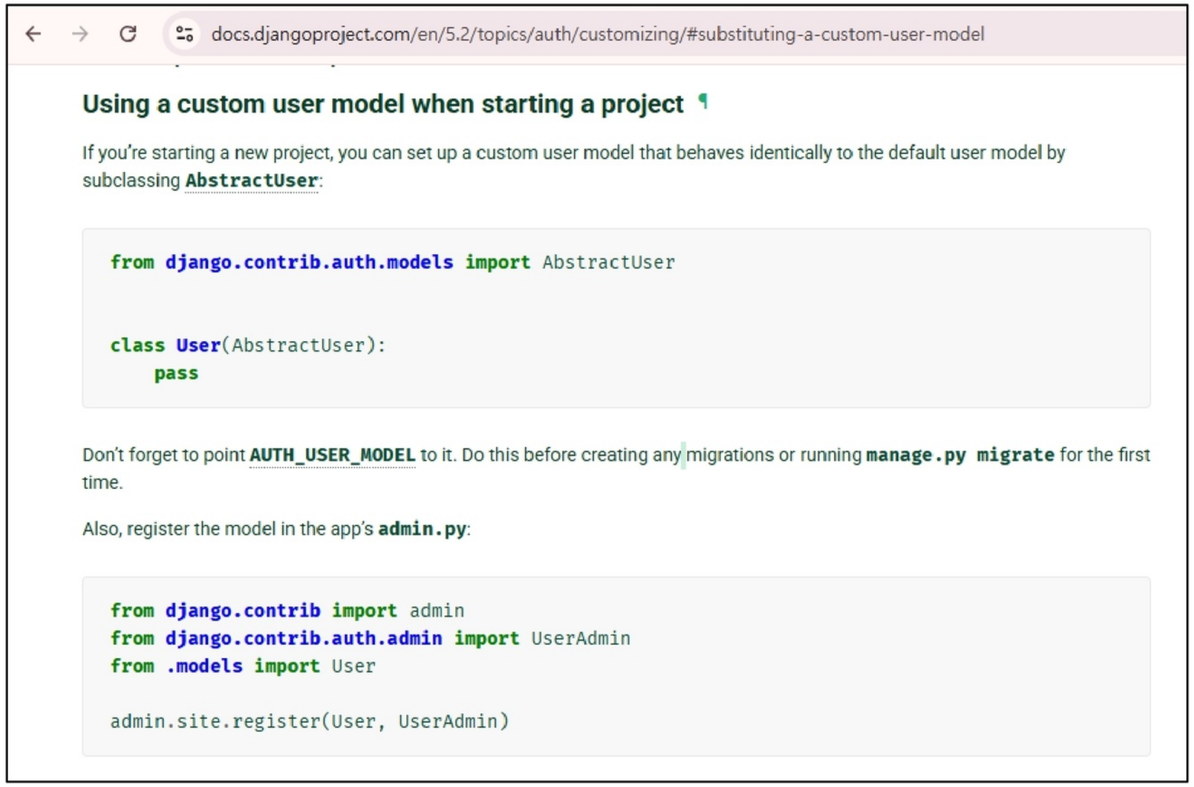
SAMSUNG SW AI ACADEMY FOR YOUTH

함께가요 미래로!
Enabling People

# User 모델 대체하기 Tip

# User 모델 대체하기 Tip

- User 모델을 대체하는 순서를 숙지하기 어려울 경우 해당 공식문서를 보며 순서대로 진행하는 것을 권장




<그림22_Django_Authentication System 01_User 모델 대체하기 Tip>

https://docs.djangoproject.com/en/5.2/topics/auth/customizing/#substituting-a-custom-user-model

# 실습



## Custom User model

- 3040. 유저 모델 대체하기
- 3243. 랭킹 관리 시스템 만들기 - 모델 정의
- 3241. 커스텀 유저 모델 구축하기

## Login

- 3041. 로그인 기능 구현하기

Test

# 확인 문제



## Q 각 문제에 올바른 답안을 생각해봅시다

**1** HTTP의 특징이 아닌 것을 고르시오.

a) 상태 유지 (Stateful)
b) 무상태 (Stateless)
c) 웹 데이터 교환의 기초
d) 비연결 지향 (Connectionless)

**2** 서버가 브라우저에 보내는 작은 데이터 조각은?

a) 캐시 (Cache)
b) 쿠키 (Cookie)
c) 세션 (Session)
d) 프로토콜 (Protocol)

**3** 쿠키의 주된 사용 목적이 아닌 것은?

a) 세션 관리
b) 사용자 개인화
c) 사용자 행동 추적
d) 데이터베이스 역할

**4** 서버 측에서 사용자 상태를 저장하는 기술은?

a) 캐시 (Cache)
b) 쿠키 (Cookie)
c) 세션 (Session)
d) 로컬 스토리지

# 확인 문제



## Q 각 문제에 올바른 답안을 생각해봅시다

### 5 세션 ID는 어디에 저장되어 전달되는가?

a) URL 파라미터
b) 서버 데이터베이스
c) 쿠키
d) 자바스크립트 변수

### 6 Django의 기본 User 모델에 없는 필드는?

a) password
b) phone_number
c) username
d) email

### 7 커스텀 User 모델 설정은 언제 해야하는가?

a) 프로젝트 중간
b) 기능 개발 완료 후
c) 데이터베이스 마이그레이션 후
d) 프로젝트 시작 시

### 8 Django에서 사용자 인증에 사용되는 폼은?

a) `CustomForm`
b) `AuthenticationForm`
c) `ModelForm`
d) `UserCreationForm`

# 확인 문제



## Q 각 문제에 올바른 답안을 생각해봅시다

### 9 로그인 성공 후 세션을 생성하는 함수는?

a) `start_session()`
b) `authenticate()`
c) `login()`
d) `session_create()`

### 10 커스텀 User 모델을 지정하는 설정 항목은?

a) `USER_MODEL_AUTH`
b) `DJANGO_USER_MODEL`
c) `CUSTOM_USER_MODEL`
d) `AUTH_USER_MODEL`

### 11 HTTP의 '무상태(stateless)'란?

a) 이전 요청 정보를 유지하지 않음
b) 서버가 클라이언트 정보를 기억함
c) 연결을 계속 유지함
d) 모든 요청은 다르게 처리됨

### 12 쿠키와 세션의 공통적인 목적은?

a) 서버 부하 감소
b) 데이터 영구 저장
c) 빠른 데이터 전송
d) 상태 정보 유지 및 사용자 식별

# 확인 문제



## 정답 및 해설

1. a) 상태 유지 (Stateful) 2. b) 쿠키 (Cookie) 3. d) 데이터베이스 역할 4. c) 세션 (Session) 5. c) 쿠키 6. b) `phone_number` 7. d) 프로젝트 시작 시
8. b) `AuthenticationForm` 9. c) login() 10. d) `AUTH_USER_MODEL` 11. a) 이전 요청 정보를 유지하지 않음 12. d) 상태 정보 유지 및 사용자 식별

1. HTTP는 상태를 유지하지 않는 무상태(Stateless) 프로토콜입니다.
2. 쿠키는 서버가 사용자의 웹 브라우저에 전송하는 작은 데이터 조각입니다.
3. 쿠키는 간단한 데이터를 저장하며, 데이터베이스를 대체할 수 없습니다.
4. 세션은 서버 측에서 클라이언트와의 상태를 유지하고 정보를 저장합니다.
5. 서버가 발급한 세션 ID는 쿠키에 저장되어 클라이언트와 서버 간에 전달됩니다.
6. 전화번호와 같은 추가 정보는 기본 User 모델에 없어 커스텀이 필요합니다.
7. Django는 프로젝트 시작 시 커스텀 User 모델 설정을 강력히 권장합니다.
8. AuthenticationForm은 사용자 로그인(인증)을 위해 사용되는 내장 폼입니다.
9. auth 모듈의 login() 함수가 인증된 사용자의 세션을 생성하고 로그인 처리합니다.
10. settings.py 파일에서 AUTH_USER_MODEL에 커스텀 모델 경로를 지정합니다.
11. 연결이 끊어지면 통신이 끝나며, 이전 요청의 상태 정보가 유지되지 않습니다.
12. 두 기술 모두 상태가 없는 HTTP에서 상태를 유지하고 사용자를 구분하기 위해 사용됩니다.

# 활동 정리



## 핵심 키워드

| 개념 | 설명 | 예시 |
|---|---|---|
| HTTP | 웹 리소스를 가져오는 규약 | 요청(Request)과 응답(Response) |
| 쿠키 (Cookie) | 브라우저에 저장하는 데이터 조각 | 장바구니, 자동 로그인 유지 |
| 세션 (Session) | 서버에 저장 및 관리되는 데이터 | 서버가 발급한 세션 ID를 쿠키에 저장 |
| Django 인증 | 사용자 인증 관련 기능 모음 | `django.contrib.auth` |
| 커스텀 유저 모델 | 기본 User 모델을 대체하는 모델 | AbstractUser 상속받아 작성 |
| AUTH_USER_MODEL | 사용할 User 모델을 지정하는 속성 | `AUTH_USER_MODEL = 'accounts.User'` |
| AuthenticationForm | 로그인 인증 데이터를 받는 Form | `form = AuthenticationForm()` |

<표1_Django_Authentication System 01_핵심 키워드>

# 활동 정리



## 오늘 공부한 내용 요약 및 정리

- HTTP (HyperText Transfer Protocol)
  - HTTP는 웹 브라우저와 서버가 데이터를 주고받기 위해 사용하는 규약
  - 이 프로토콜은 두 가지 중요한 특징을 가짐
  - 비연결 지향 (Connectionless)
    - 서버는 클라이언트의 요청에 응답한 후 연결을 끊음
    - 이는 수많은 사용자와의 연결을 계속 유지할 때 발생하는 서버 자원 낭비를 막기 위함
  - 무상태 (Stateless)
    - 연결이 끊어지면 서버는 클라이언트의 이전 상태 정보를 저장하지 않음
    - 이 때문에 로그인 상태나 장바구니 정보가 유지되지 않는 문제가 발생

# 활동 정리



## 오늘 공부한 내용 요약 및 정리

- 쿠키 (Cookie)
  - 쿠키는 서버가 사용자의 웹 브라우저에 보내는 작은 데이터 조각으로, key-value 형식으로 저장
  - 이를 통해 서버는 사용자를 식별하고 로그인 유지, 장바구니, 사용자 설정 기억 등의 상태를 유지할 수 있음
  - 동작 방식
    1. 서버는 응답 시 Set-Cookie 헤더를 통해 쿠키를 브라우저에 보냄
    2. 브라우저는 이 쿠키를 저장했다가, 같은 서버에 다시 요청할 때마다 저장된 쿠키를 함께 전송
    3. 서버는 쿠키 정보를 확인하여 사용자를 식별

# 활동 정리



## 오늘 공부한 내용 요약 및 정리

- 세션 (Session)
  - 세션은 로그인 정보와 같은 중요한 데이터를 클라이언트가 아닌 서버 측에 저장하는 기술
  - 실제 데이터는 서버에 보관되므로 쿠키만 사용하는 방식보다 보안에 유리
  - 동작 방식
    1. 사용자가 로그인에 성공하면 서버는 세션 데이터를 생성하고 고유한 session id를 발급
    2. 서버는 이 session id만 쿠키에 담아 클라이언트에게 보냄
    3. 클라이언트는 이후 요청마다 session id가 담긴 쿠키를 서버로 전송하여 인증 상태를 유지

# 활동 정리



## 오늘 공부한 내용 요약 및 정리

- Django 인증 시스템
  - Django의 인증 시스템은 사용자 인증과 관련된 핵심 기능을 제공
  - 하지만 기본으로 제공되는 User 모델은 필드가 제한적이고, 생년월일이나 주소 같은 추가 정보를 다루기 어려움
  - Custom User Model
    - 프로젝트 시작 단계에서 Django의 기본 User 모델을 직접 만든 모델로 대체하는 것이 강력히 권장
    - 나중에 필드를 추가하는 등 유연하게 확장할 수 있음

# 활동 정리



## 오늘 공부한 내용 요약 및 정리

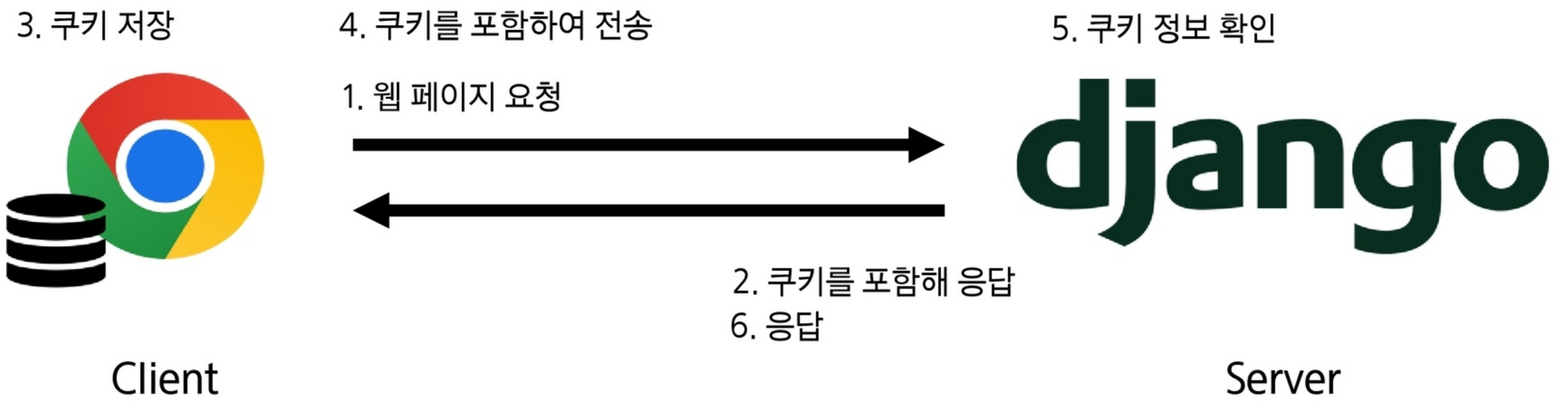
- 로그인 기능 구현
  - 로그인은 사용자가 자신임을 서버에 인증하고, 서버가 세션을 만들어 클라이언트와 연결하는 과정
  - 구현 절차
    1. 로그인 데이터를 입력받기 위해 Django에 내장된 AuthenticationForm을 사용함
       (이 폼은 모델과 직접 연결되지 않고 오직 "인증" 역할만 수행함)
    2. 사용자가 정보를 제출하면(POST), 폼의 유효성을 검사함
    3. 유효하다면 auth_login 함수를 호출하여 사용자를 로그인시키고, 세션 데이터를 생성하여 django_session 테이블에 저장함
    4. 이때 발급된 sessionid는 브라우저의 쿠키에 저장되어 사용자의 로그인 상태를 유지하는 데 사용됨

# 활동 정리



## 매번 방문하는 웹사이트, 어떻게 나를 기억하고 반겨줄까요?

1. 우리가 즐겨 찾는 쇼핑몰이나 SNS에 접속할 때, 아이디와 비밀번호를 한 번만 입력하면 다음 방문 시에는 자동으로 로그인 상태가 유지됩니다.
2. 하지만 웹은 '무상태' 특징을 가지고 있습니다. 추가로 '비연결' 특징도 가지고 있습니다.페이지를 이동할 때마다 우리는 사실 '처음 보는 사람' 취급을 받게 됩니다.
3. 그렇다면 웹사이트는 어떻게 이 문제를 해결하고 우리를 기억해 주는 걸까요?




<그림23_Django_Authentication System 01_쿠키 동작 예시 종합>

SAMSUNG SW AI ACADEMY FOR YOUTH

함께가요 미래로!
Enabling People

# 감사합니다